

后端开发常问面试题集锦

——算法问题



欢迎做 Java 的工程师朋友们加入 Java 高级架构进阶群；群内有技术大咖指点难题，还提供免费的 Java 架构学习资料（里面有高可用,高并发,高性能及分布式,Jvm 性能调优,Spring 源码,MyBatis,Netty,Redis,Kafka,Mysql,Zookeeper,Tomcat,Docker,Dubbo,Nginx 等多个知识点的架构资料）

群名称:Java高级架构进阶@群
群 号:963944895

五分钟理解一致性哈希算法

一致性哈希算法在 1997 年由麻省理工学院提出的一种分布式哈希（DHT）实现算法，设计目标是为了解决因特网中的热点 (Hot spot) 问题，初衷和 CARP 十分类似。一致性哈希修正了 CARP 使用的简单哈希算法带来的问题，使得分布式哈希（DHT）可以在 P2P 环境中真正得到应用。

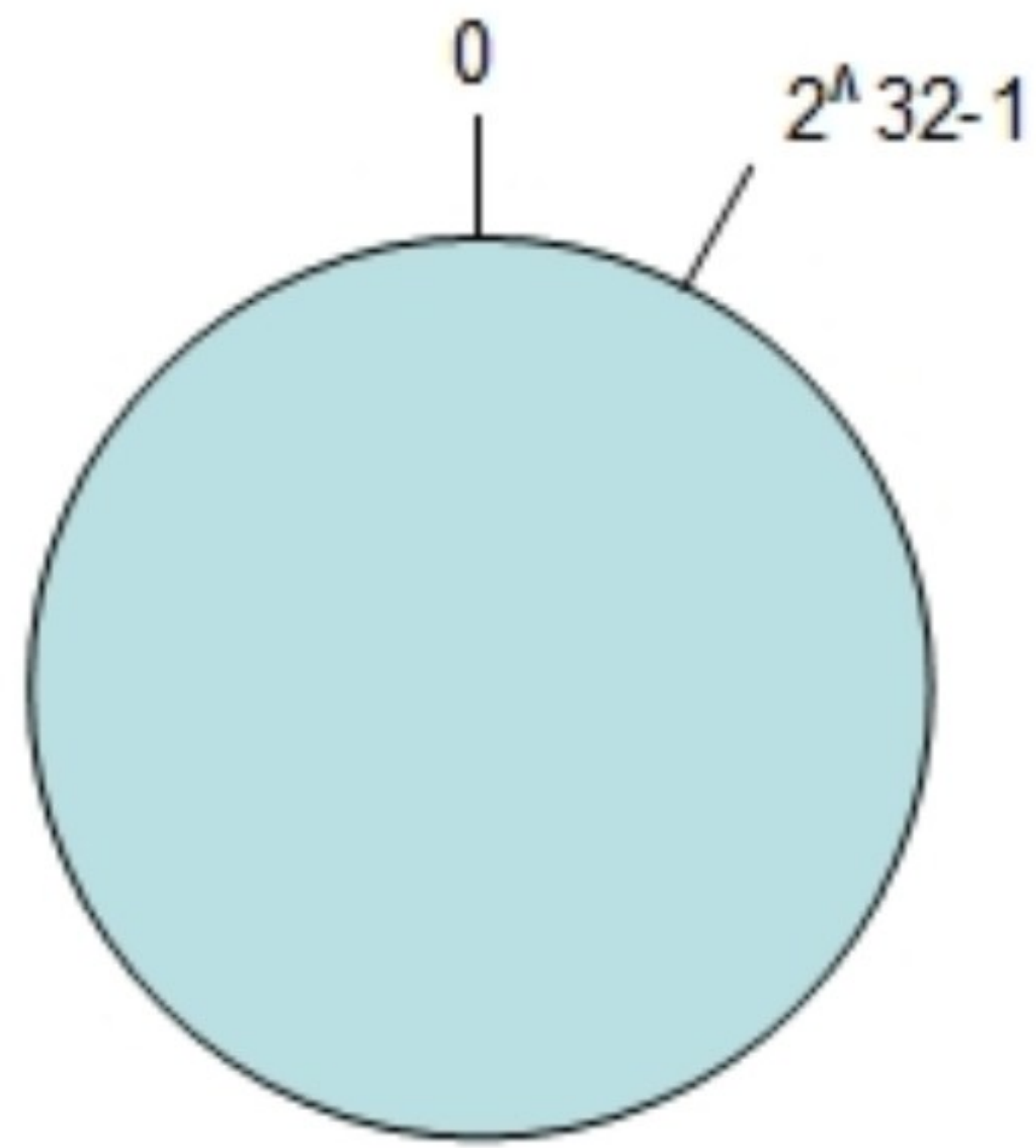
一致性 hash 算法提出了在动态变化的 Cache 环境中，判定哈希算法好坏的四个定义：

- 1、**平衡性 (Balance)**：平衡性是指哈希的结果能够尽可能分布到所有的缓冲中去，这样可以使得所有的缓冲空间都得到利用。很多哈希算法都能够满足这一条件。
- 2、**单调性 (Monotonicity)**：单调性是指如果已经有一些内容通过哈希分派到了相应的缓冲中，又有新的缓冲加入到系统中。哈希的结果应能够保证原有已分配的内容可以被映射到原有的或者新的缓冲中去，而不会被映射到旧的缓冲集合中的其他缓冲区。
- 3、**分散性 (Spread)**：在分布式环境中，终端有可能看不到所有的缓冲，而是只能看到其中的一部分。当终端希望通过哈希过程将内容映射到缓冲上时，由于不同终端所见的缓冲范围有可能不同，从而导致哈希的结果不一致，最终的结果是相同的内容被不同的终端映射到不同的缓冲区中。这种情况显然是应该避免的，因为它导致相同内容被存储到不同缓冲中去，降低了系统存储的效率。分散性的定义就是上述情况发生的严重程度。好的哈希算法应能够尽量避免不一致的情况发生，也就是尽量降低分散性。
- 4、**负载 (Load)**：负载问题实际上是从另一个角度看待分散性问题。既然不同的终端可能将相同的内容映射到不同的缓冲区中，那么对于一个特定的缓冲区而言，也可能被不同的用户映射为不同的内容。与分散性一样，这种情况也是应当避免的，因此好的哈希算法应能够尽量降低缓冲的负荷。

在分布式集群中，对机器的添加删除，或者机器故障后自动脱离集群这些操作是分布式集群管理最基本的功能。如果采用常用的 $\text{hash}(\text{object})\%N$ 算法，那么在有机器的添加或者删除后，很多原有的数据就无法找到了，这样严重的违反了单调性原则。接下来主要讲解一下一致性哈希算法是如何设计的：

环形 Hash 空间

按照常用的 hash 算法来将对应的 key 哈希到一个具有 2^{32} 次方个桶的空间中，即 $0 \sim (2^{32}) - 1$ 的数字空间中。现在我们可以将这些数字头尾相连，想象成一个闭合的环形。如下图



把数据通过一定的 hash 算法处理后映射到环上

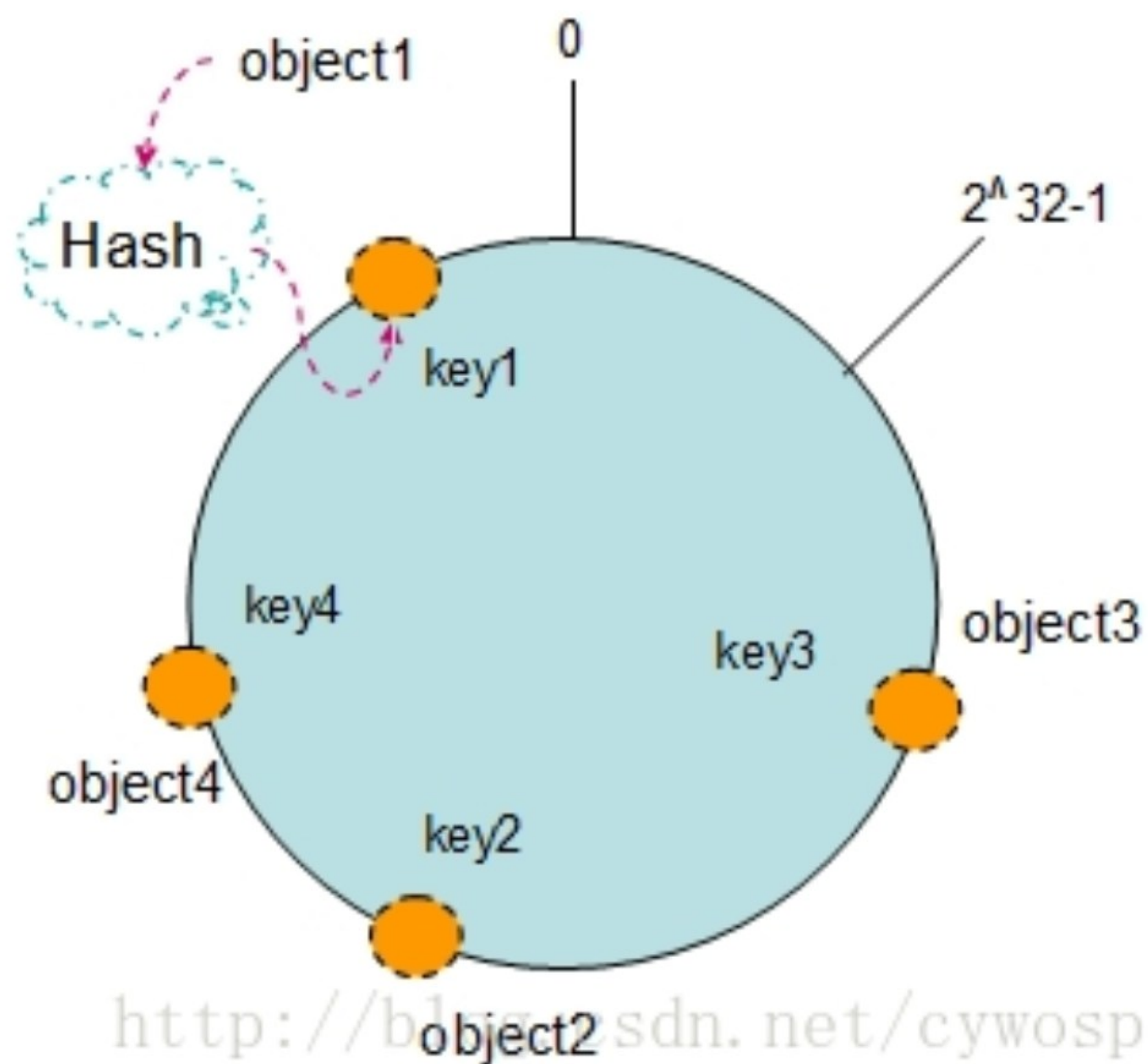
现在我们将 object1、object2、object3、object4 四个对象通过特定的 Hash 函数计算出对应的 key 值，然后散列到 Hash 环上。如下图：

$\text{Hash}(\text{object1}) = \text{key1};$

$\text{Hash}(\text{object2}) = \text{key2};$

$\text{Hash}(\text{object3}) = \text{key3};$

$\text{Hash}(\text{object4}) = \text{key4};$



将机器通过 hash 算法映射到环上

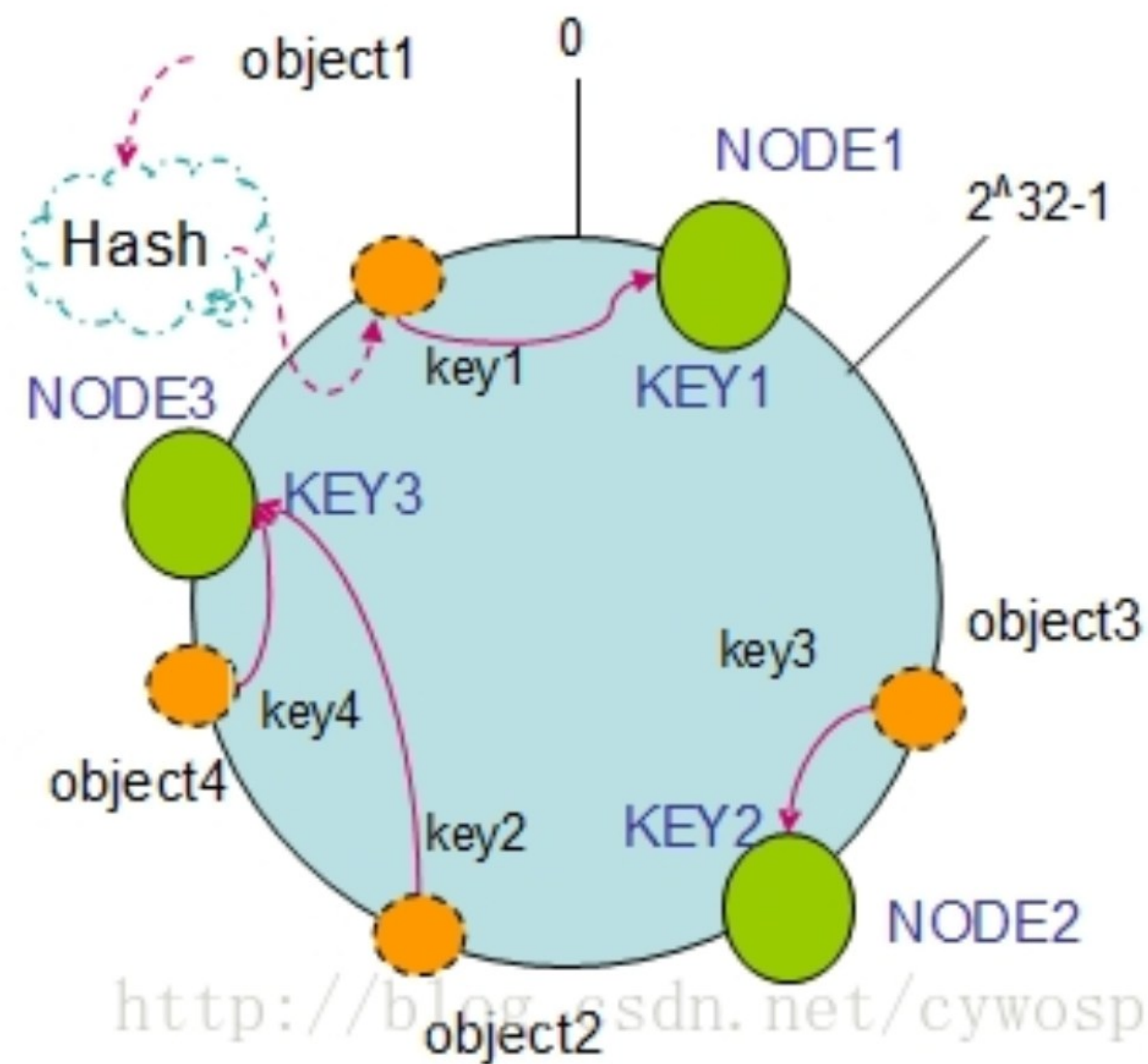
在采用一致性哈希算法的分布式集群中将新的机器加入，其原理是通过使用与对象存储一样的 Hash 算法将机器也映射到环中（一般情况下对机器的 hash 计算是采用机器的 IP 或者机器唯一的别名作为输入值），然后以顺时针的方向计算，将所有对象存储到离自己最近的机器中。

假设现在有 NODE1，NODE2，NODE3 三台机器，通过 Hash 算法得到对应的 KEY 值，映射到环中，其示意图如下：

$\text{Hash}(\text{NODE1}) = \text{KEY1};$

$\text{Hash}(\text{NODE2}) = \text{KEY2};$

$\text{Hash}(\text{NODE3}) = \text{KEY3};$



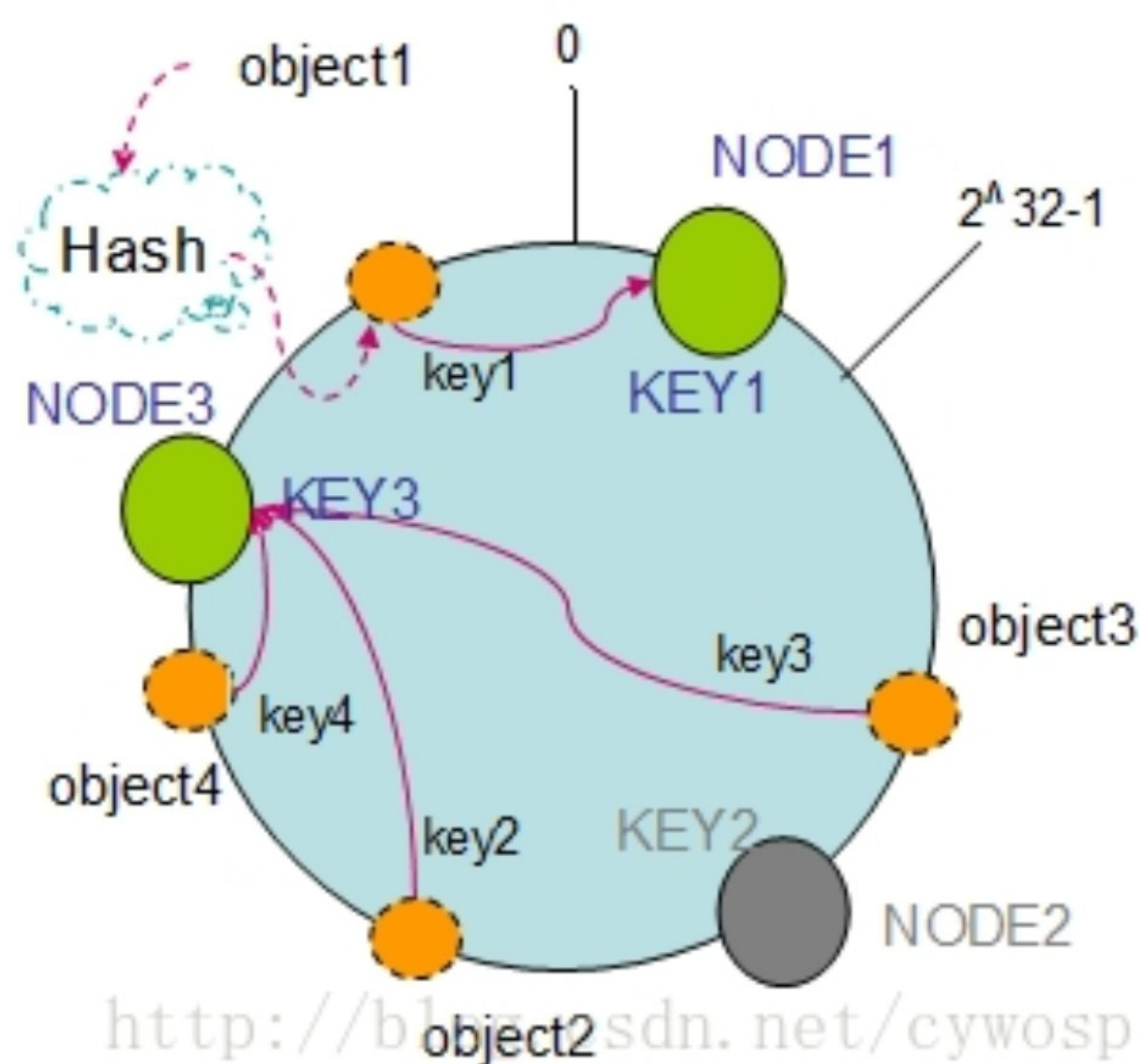
通过上图可以看出对象与机器处于同一哈希空间中，这样按顺时针转动 object1 存储到了 NODE1 中，object3 存储到了 NODE2 中，object2、object4 存储到了 NODE3 中。在这样的部署环境中，hash 环是不会变更的，因此，通过算出对象的 hash 值就能快速的定位到对应的机器中，这样就能找到对象真正的存储位置了。

机器的删除与添加

普通 hash 求余算法最为不妥的地方就是在有机器的添加或者删除之后会照成大量的对象存储位置失效，这样就大大的不满足单调性了。下面来分析一下一致性哈希算法是如何处理的。

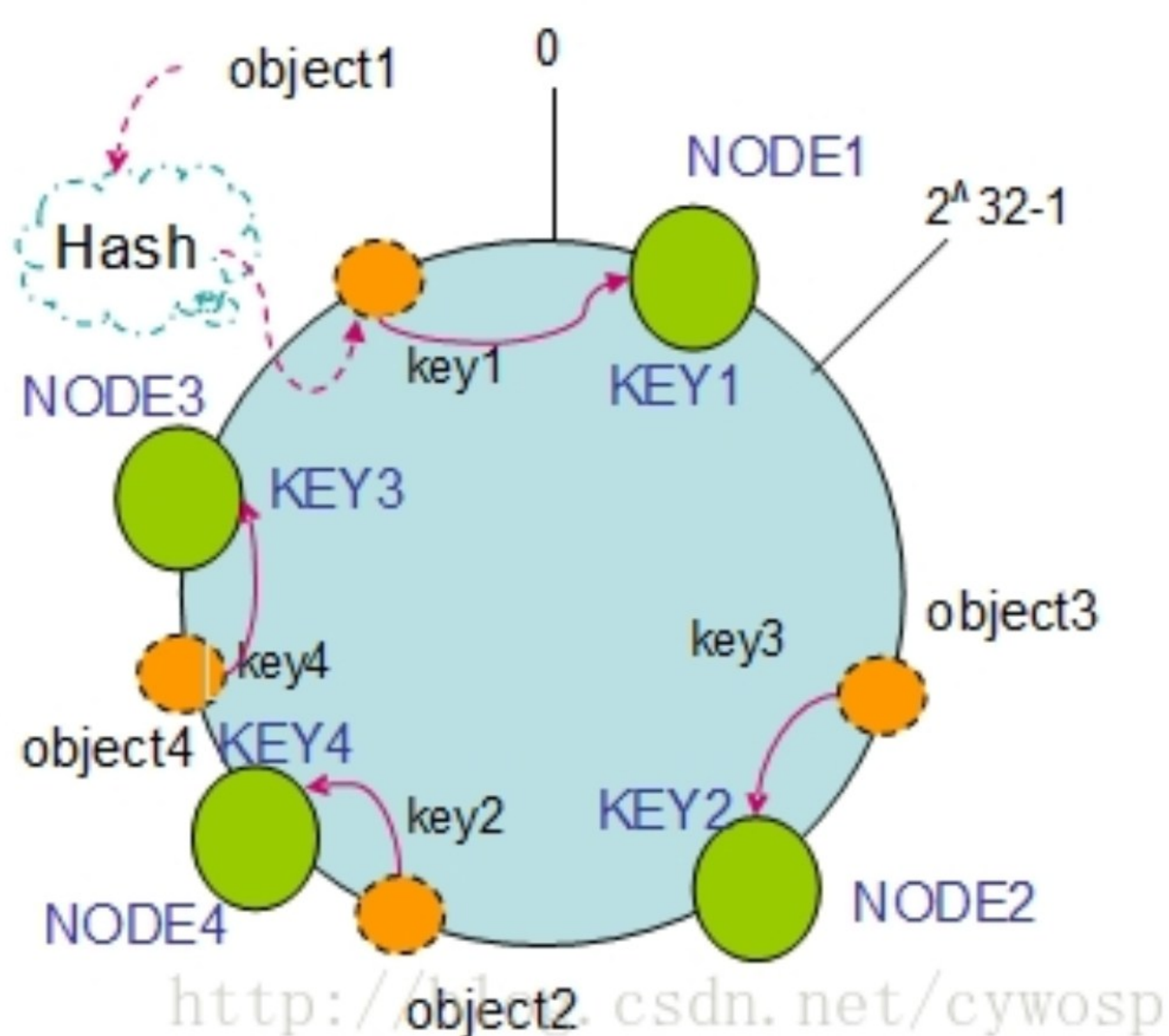
1. 节点（机器）的删除

以上面的分布为例，如果 NODE2 出现故障被删除了，那么按照顺时针迁移的方法，object3 将会被迁移到 NODE3 中，这样仅仅是 object3 的映射位置发生了变化，其它的对象没有任何的改动。如下图：



2. 节点（机器）的添加

如果往集群中添加一个新的节点 NODE4，通过对应的哈希算法得到 KEY4，并映射到环中，如下图：



通过按顺时针迁移的规则，那么 object2 被迁移到了 NODE4 中，其它对象还保持这原有的存储位置。通过对节点的添加和删除的分析，一致性哈希算法在保持了单调性的同时，还是数据的迁移达到了最小，这样的算法对分布式集群来说是非常合适的，避免了大量数据迁移，减小了服务器的压力。

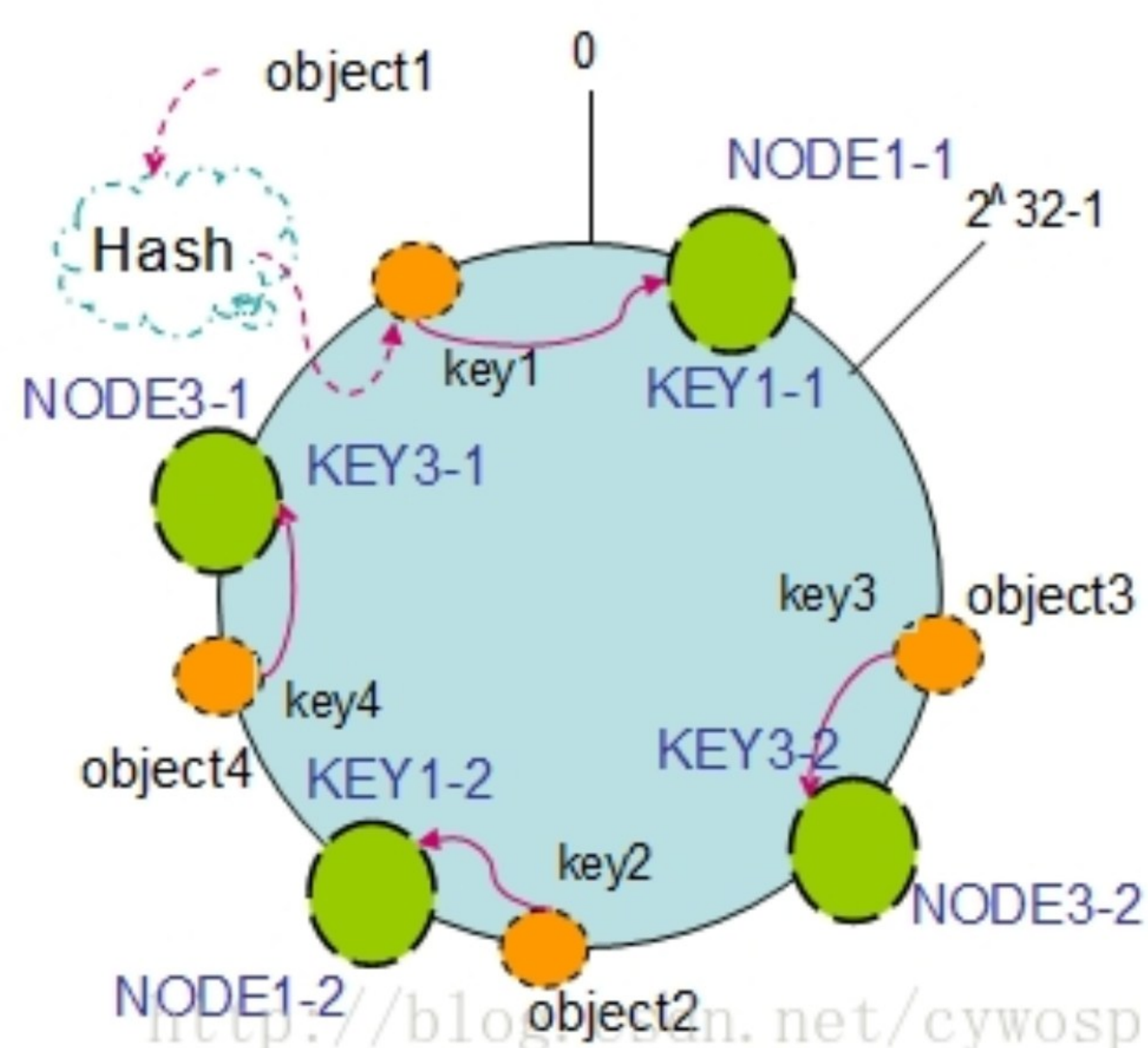
平衡性

根据上面的图解分析，一致性哈希算法满足了单调性和负载均衡的特性以及一般 hash 算法的分散性，但这还并不能当做其被广泛应用的原由，因为还缺少了

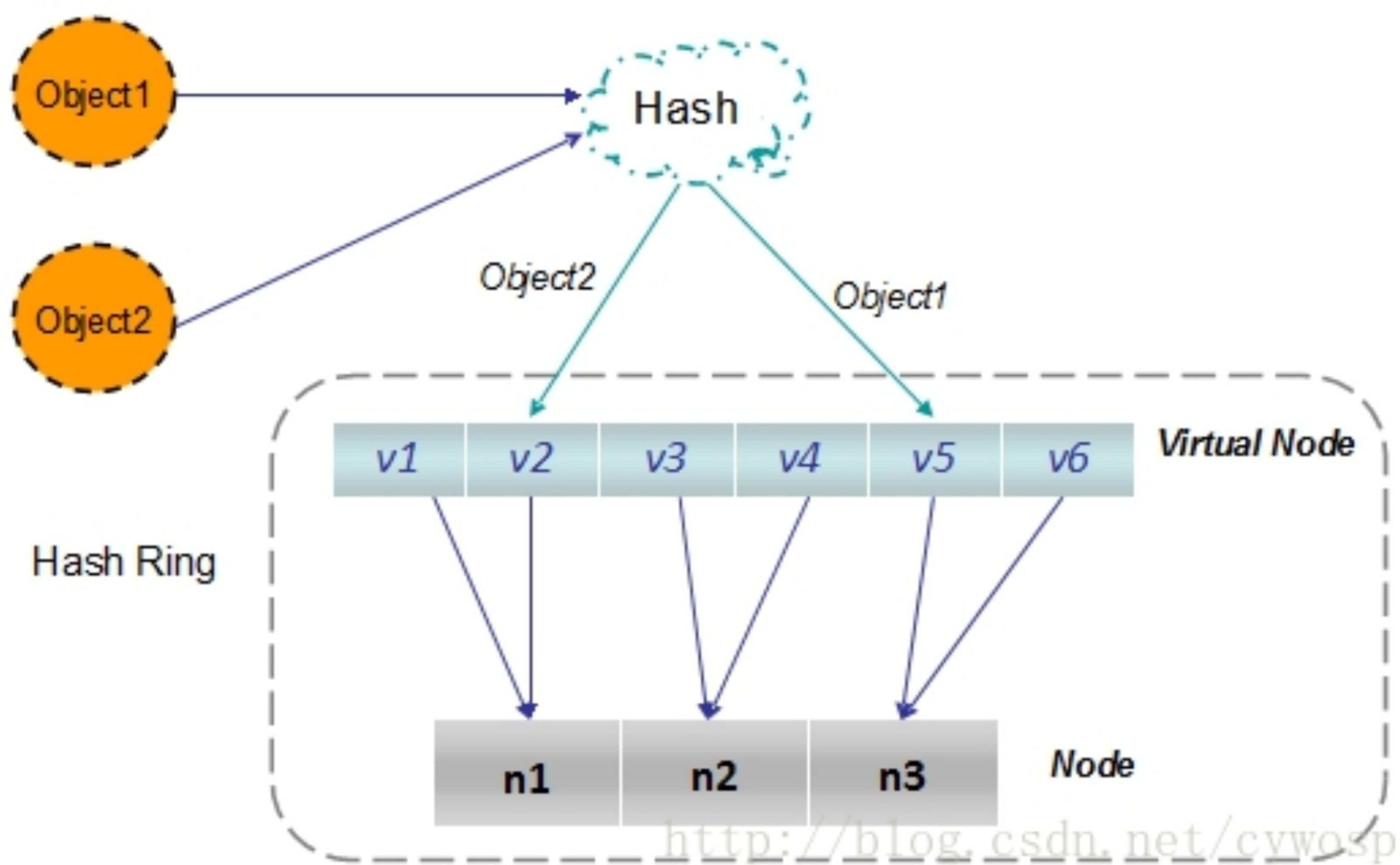
平衡性。下面将分析一致性哈希算法是如何满足平衡性的。hash 算法是不保证平衡的，如上面只部署了 NODE1 和 NODE3 的情况（NODE2 被删除的图），object1 存储到了 NODE1 中，而 object2、object3、object4 都存储到了 NODE3 中，这样就造成了非常不平衡的状态。在一致性哈希算法中，为了尽可能的满足平衡性，其引入了虚拟节点。

——“虚拟节点”（virtual node）是实际节点（机器）在 hash 空间的复制品（replica），一实际个节点（机器）对应了若干个“虚拟节点”，这个对应个数也成为“复制个数”，“虚拟节点”在 hash 空间中以 hash 值排列。

以上面只部署了 NODE1 和 NODE3 的情况（NODE2 被删除的图）为例，之前的对象在机器上的分布很不均衡，现在我们以 2 个副本（复制个数）为例，这样整个 hash 环中就存在了 4 个虚拟节点，最后对象映射的关系图如下：



根据上图可知对象的映射关系：object1->NODE1-1，object2->NODE1-2，object3->NODE3-2，object4->NODE3-1。通过虚拟节点的引入，对象的分布就比较均衡了。那么在实际操作中，正真的对象查询是如何工作的呢？对象从 hash 到虚拟节点到实际节点的转换如下图：



“虚拟节点”的 hash 计算可以采用对应节点的 IP 地址加数字后缀的方式。例如假设 NODE1 的 IP 地址为 192.168.1.100。引入“虚拟节点”前，计算 cache A 的 hash 值：

```
Hash(“192.168.1.100”);
```

引入“虚拟节点”后，计算“虚拟节点”点 NODE1-1 和 NODE1-2 的 hash 值：

```
Hash(“192.168.1.100#1”); // NODE1-1
```

```
Hash(“192.168.1.100#2”); // NODE1-2
```